PROGRAMA PARA AJUSTE DE CURVAS

Arquitetura de Software e Baseline de Desenvolvimento

Aplicação desktop para curve fitting com modelos arbitrários definidos em LaTeX

Documento	Architecture Baseline
Versão	1.4
Status	Baseline revisada - Dataset matricial canônico e validação simplificada aprovados
Data	12/10/2026
Escopo	Arquitetura, pacotes, objetos, fluxo, fitting, GUI, testes e conformidade

Este documento é a referência arquitetural normativa do projeto. Alterações estruturais relevantes devem ser confrontadas com esta baseline antes de serem incorporadas.

Sumário

1. Objetivo e escopo
2. Princípios arquiteturais
3. Requisitos funcionais e não funcionais
4. Arquitetura geral
5. Estrutura de pastas e pacotes
6. Modelo de domínio
7. Data Engine
8. Model Engine e interpretação de LaTeX
9. Fitting Engine
10. Modelos padrão
11. Interface gráfica Tkinter
12. Resultados, gráficos e métricas
13. Tratamento de erros e segurança
14. Estratégia de testes
15. Dependências e tecnologias
16. Fluxos principais
17. Roadmap de implementação
18. Regras de conformidade e controle de mudanças
19. Checklist de verificação
20. Arquitetura completa do software
21. Decisões em aberto e extensões futuras
22. Referências técnicas

1. Objetivo e escopo

O objetivo é desenvolver uma aplicação desktop em Python, com interface gráfica em Tkinter e arquitetura orientada a objetos, capaz de ajustar conjuntos de dados experimentais a modelos matemáticos conhecidos ou a um modelo arbitrário informado pelo usuário em LaTeX. A V1 opera com exatamente uma variável independente e uma dependente, mas os contratos centrais usam coleções de Variable e matrizes NumPy 2D para permitir evolução futura para múltiplas variáveis sem redesenhar o domínio.

- Importar dados a partir de CSV e Excel.
- Receber dados por copiar/colar, reconhecendo separadores por vírgula, espaço e tabulação.
- Disponibilizar modelos frequentes, como linear, polinomial, exponencial, logarítmico, potência, gaussiano e sigmoidal.
- Receber uma equação customizada na forma variável_dependente = f(variável_independente, parâmetros) em LaTeX, usando x e y apenas como símbolos padrão, não obrigatórios.
- Interpretar a equação simbolicamente, associar os símbolos configurados às variáveis independente e dependente e extrair os demais símbolos livres como parâmetros de ajuste.
- Permitir valores iniciais, limites e fixação/liberação de parâmetros.
- Executar o ajuste numérico, avaliar a qualidade do fit e retornar parâmetros, incertezas e métricas.
- Exibir pontos experimentais e curva ajustada em gráfico incorporado à interface.

Fora do escopo funcional inicial: regressões implícitas F(x,y,...)=0, fitting efetivo com múltiplas variáveis independentes/dependentes, inferência bayesiana, fitting global de múltiplos datasets e processamento distribuído. Os contratos de Dataset e ModelDefinition, porém, devem permanecer preparados para a evolução multidimensional.

2. Princípios arquiteturais

Princípio	Diretriz
Separação GUI / domínio	Tkinter não deve conter a lógica científica. Callbacks delegam a controllers/services.
Representação interna simbólica	LaTeX é formato de entrada/apresentação; internamente o modelo é uma expressão SymPy validada.
Dependência por interfaces	DataLoader e FitEngine devem possuir contratos abstratos para permitir substituições futuras.
Objetos de domínio estáveis	Dataset, Variable, Parameter, ModelDefinition e FitResult representam conceitos científicos independentes da GUI.
Uma única pipeline de modelos	Modelos built-in e customizados devem convergir para ModelDefinition e usar o mesmo motor de fitting.
Segurança por desenho	Não executar eval() de entrada do usuário. Parsing, whitelist de funções e validação antecedem a geração numérica.
Testabilidade	Núcleo matemático deve operar sem iniciar Tkinter, permitindo testes unitários e integração em modo headless.
Baseline evolutiva	Mudanças arquiteturais são permitidas, mas devem

Princípio	Diretriz
	gerar decisão explícita: adaptar o código ou revisar este documento.

3. Requisitos funcionais e não funcionais

3.1 Requisitos funcionais

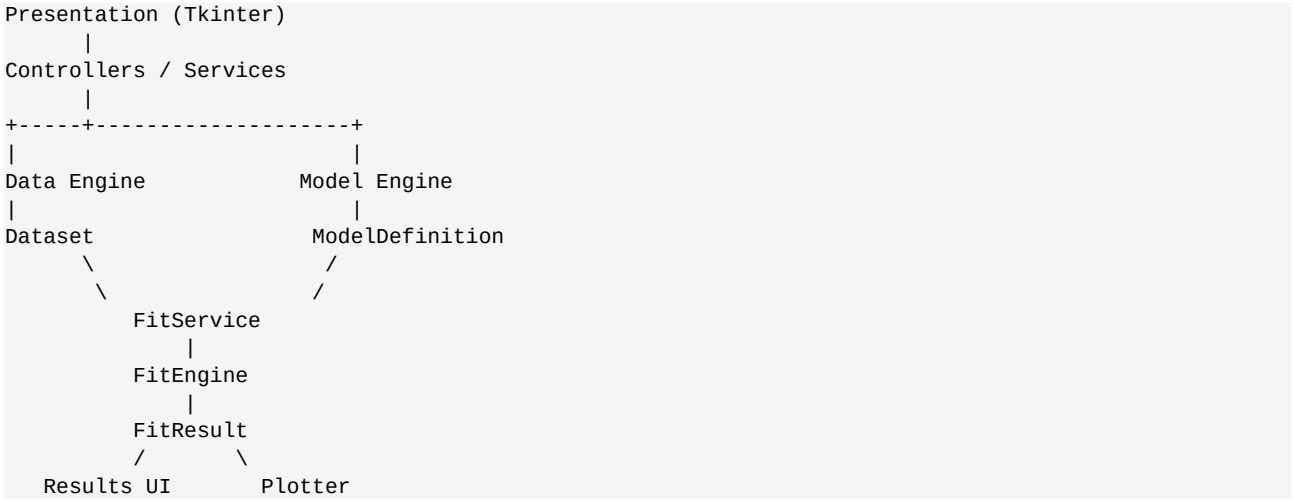
ID	Requisito
RF-01	Importar CSV com seleção das colunas associadas às variáveis independente e dependente; a V1 seleciona uma de cada, convertidas para matrizes NumPy 2D.
RF-02	Importar XLSX/XLS, selecionar planilha e colunas e converter os dados selecionados para a representação matricial canônica do Dataset.
RF-03	Colar dados tabulares e detectar delimitador/decimal.
RF-04	Editar/limpar a tabela de dados.
RF-05	Selecionar modelo built-in.
RF-06	Inserir modelo customizado explícito em LaTeX, com símbolos configuráveis para variável independente e dependente.
RF-07	Validar e apresentar a interpretação do modelo antes do fit.
RF-08	Extrair parâmetros automaticamente, excluindo a variável independente configurada e constantes reconhecidas.
RF-09	Editar name, unit, initial_value, min_value, max_value e vary de cada parâmetro; symbol é a identidade matemática imutável. Reconfiguração bem-sucedida deve invalidar o resultado de fit anterior.
RF-10	Executar fitting e interromper com mensagem controlada em caso de falha.
RF-11	Exibir parâmetros ajustados e incertezas.
RF-12	Calcular e exibir métricas de ajuste.
RF-13	Exibir pontos e curva ajustada em Matplotlib.
RF-14	Permitir novo fit sem reiniciar a aplicação.
RF-15	Permitir configurar symbol, name e unit das variáveis independente e dependente, usando x/y apenas como defaults.

3.2 Requisitos não funcionais

ID	Requisito
RNF-01	Arquitetura orientada a objetos e modular.
RNF-02	GUI responsiva; cálculos potencialmente demorados não devem congelar permanentemente a interface.
RNF-03	Mensagens de erro compreensíveis e sem traceback bruto para o usuário final.
RNF-04	Entrada matemática não pode executar código Python arbitrário.
RNF-05	Cobertura de testes prioritária para parser, validação e fitting.
RNF-06	Reprodutibilidade de resultados dentro das tolerâncias numéricas definidas.
RNF-07	Componentes matemáticos reutilizáveis fora do Tkinter.
RNF-08	Código tipado progressivamente e documentado.

4. Arquitetura geral

A aplicação será organizada em camadas. A Presentation Layer captura eventos e exibe estado; controllers traduzem eventos em operações de aplicação; services orquestram objetos de domínio e engines; os engines executam importação, parsing, fitting e plotting.



Regra central: A GUI nunca deve manipular diretamente expressões SymPy, chamar scipy/lmfit diretamente ou interpretar arquivos. Ela trabalha por meio dos controllers/services.

5. Estrutura de pastas e pacotes

```
curve_fitting_app/
|-- pyproject.toml
|-- README.md
|-- requirements.txt
|-- src/
|   `-- curvefit/
|       |-- __init__.py
|       |-- main.py
|       |-- domain/
|       |   |-- validation/
|       |   |   |-- __init__.py
|       |   |   |-- numeric.py
|       |   |   |-- text.py
|       |   |   `-- symbol.py
|       |   |-- parameter/
|       |   |   |-- __init__.py
|       |   |   |-- parameter.py
|       |   |   `-- validator.py
|       |   |-- variable/
|       |   |   |-- __init__.py
|       |   |   |-- variable.py
|       |   |   `-- validator.py
|       |   |-- dataset/
|       |   |   |-- __init__.py
|       |   |   |-- dataset.py
|       |   |   `-- validator.py
|       |   |-- model.py
|       |   |-- fit_result.py
|       |   `-- exceptions.py
|       |-- data/
|       |   |-- data_loader.py
|       |   |-- csv_loader.py
|       |   |-- excel_loader.py
|       |   |-- clipboard_parser.py
|       |   `-- data_validator.py
|       |-- models/
|       |   |-- model_registry.py
|       |   |-- builtin_models.py
|       |   |-- custom_model.py
|       |   `-- latex_parser.py
|       |-- fitting/
|       |   |-- fit_engine.py
|       |   |-- lmfit_engine.py
|       |   |-- statistics.py
|       |   `-- initial_guess.py
|       |-- services/
|       |   |-- data_service.py
|       |   |-- model_service.py
|       |   `-- fit_service.py
|       |-- gui/
|       |   |-- app.py
|       |   |-- controllers/
|       |   |-- views/
|       |   `-- widgets/
|       `-- plotting/
|           `-- plotter.py
|-- tests/
|   |-- unit/
|   |   |-- domain/
|   |   |-- data/
|   |   |-- models/
|   |   |-- fitting/
|   |   `-- gui/
|   |-- integration/
|   `-- fixtures/
`-- examples/
```

6. Modelo de domínio

6.1 Dataset

Representa um conjunto de dados experimentais já validado e pronto para consumo científico. Não conhece CSV, Excel, Pandas ou Tkinter. A representação numérica canônica usa matrizes NumPy bidimensionais: cada linha representa uma observação experimental e cada coluna representa uma Variable. Mesmo quando existe apenas uma variável, a matriz mantém shape (n_samples, 1).

```
Dataset
- independent_variables: tuple[Variable, ...]
- dependent_variables: tuple[Variable, ...]
- independent_data: numpy.ndarray  # shape (n_samples, n_independent_variables)
- dependent_data: numpy.ndarray    # shape (n_samples, n_dependent_variables)
- metadata: dict

Contrato canônico:
- independent_data e dependent_data são sempre matrizes 2D de float64.
- as linhas representam observações correspondentes; independent_data e dependent_data devem ter o
mesmo número de linhas.
- as colunas correspondem, por posição, às Variable de independent_variables e dependent_variables.
- a API pública pode receber uma única Variable ou uma Sequence[Variable];
DatasetValidator.variables(...) normaliza ambas para tuple[Variable, ...].
- a V1 impõe exatamente uma variável independente e uma dependente como política de versão, não como
limitação estrutural.
- Pandas é permitido na camada Data para importação/manipulação; a conversão DataFrame -> ndarray
ocorre antes da construção do Dataset.

DatasetValidator usa mensagens genéricas nos validadores reutilizáveis e não recebe field_name
apenas para personalizar erros. O contexto independente/dependente aparece somente nas validações
relacionais em que é semanticamente necessário.

DatasetValidator possui quatro responsabilidades específicas: variables(...) normaliza uma Variable
ou Sequence[Variable] para tuple e valida cardinalidade/símbolos; data_matrix(...) valida e copia
matrizes 2D numéricas finitas em float64; matching_variable_count(...) garante uma coluna por
Variable; matching_sample_count(...) e distinct_symbols(...) validam as relações entre entradas e
saídas. Os métodos genéricos usam mensagens de erro neutras, evitando parâmetros field_name que não
alteram a regra de validação.
```

6.2 Parameter

```
Parameter
- symbol: str
- name: str | None
- initial_value: float
- value: float | None
- min_value: float
- max_value: float
- vary: bool
- stderr: float | None
- unit: str | None

Semântica:
- symbol: identidade matemática imutável do parâmetro. Deve representar um símbolo matemático
atômico válido segundo a gramática restrita da aplicação; aceita identificadores simples, letras
gregas Unicode e comandos LaTeX gregos reconhecidos, com um subscrito simples opcional. Expressões
completas não são símbolos.
- name: descrição humana opcional para GUI, títulos, legendas e relatórios.
- unit: unidade física opcional.
- value e stderr: resultado do último fitting válido; não fazem parte da configuração de entrada.

Defaults de configuração:
- DEFAULT_NAME = None
- DEFAULT_INITIAL_VALUE = 1.0
- DEFAULT_MIN_VALUE = -inf
- DEFAULT_MAX_VALUE = +inf
- DEFAULT_VARY = True
- DEFAULT_UNIT = None

Contrato comportamental:
- configure(...) substitui a configuração completa do parâmetro; argumentos omitidos assumem os
defaults da classe.
- configure() sem argumentos restaura todos os defaults de configuração, preservando exclusivamente
symbol.
- qualquer configure(...) bem-sucedido invalida e limpa value/stderr do fit anterior.
- clear_fit_result() limpa somente value e stderr, preservando symbol, name, unit e configuração de
fitting.
- alterações são validadas integralmente antes da mutação do estado; uma configuração inválida não
deve deixar o objeto parcialmente alterado.
- symbol não pode ser alterado por configure(); mudar a identidade matemática implica criar outro
Parameter.

ParameterValidator delega a sintaxe atômica de symbol a SymbolValidator; a interpretação sintática e
semântica de expressões LaTeX completas pertence ao Model Engine.

Composição de validação:
- TextValidator centraliza validação/normalização de texto obrigatório e opcional.
- SymbolValidator centraliza a gramática de símbolos matemáticos atômicos compartilhada por
Parameter e Variable.
- ParameterValidator compõe NumericValidator, TextValidator e SymbolValidator e expõe as regras
específicas de Parameter (symbol, name, unit, bounds, initial_value, fit_value, stderr e vary).
- Parameter conhece apenas ParameterValidator; não depende diretamente dos validadores genéricos.
- A validação de SymbolValidator é deliberadamente sintática e restrita; a interpretação de
expressões LaTeX completas e conflitos semânticos com funções/constantes pertencem ao Model Engine.
```

6.3 Variable

```
Variable
- symbol: str
- name: str | None
- unit: str | None

Variable representa uma grandeza observada do dataset, e não um parâmetro de otimização. O mesmo
tipo é usado para a variável independente e para a variável dependente. x e y são apenas defaults de
símbolo. Exemplos: t / Tempo / s; T / Temperatura / °C.
```

6.4 ModelDefinition

```
ModelDefinition
- name: str
- latex_expression: str
- symbolic_expression: sympy.Expr
- independent_variables: tuple[Variable, ...]
- dependent_variables: tuple[Variable, ...]
- parameters: list[Parameter]
- numeric_function: callable
- metadata: dict

A V1 aceita exatamente uma variável de cada coleção, mas o contrato usa o plural desde o início para
preservar a evolução multidimensional.
```

6.5 FitResult

```
FitResult
- success: bool
- message: str
- parameters: dict[str, Parameter]
- covariance: ndarray | None
- residuals: ndarray
- fitted_y: ndarray
- x_smooth: ndarray
- y_smooth: ndarray
- rss: float
- rmse: float
- r_squared: float
- reduced_chi_square: float | None
- metadata: dict
```

7. Data Engine

O Data Engine converte diferentes origens para um Dataset coerente. A interface DataLoader define o contrato comum. Implementações concretas tratam formatos específicos, enquanto validações de importação/limpeza permanecem na camada Data. Pandas pode ser usado para leitura, seleção e limpeza tabular; antes de entrar no domínio, as colunas selecionadas são convertidas para matrizes NumPy 2D float64 no formato (n_samples, n_variables). Dataset e DatasetValidator não dependem de Pandas.

- CSVLoader: leitura via Pandas, detecção/seleção de delimitador, cabeçalho e colunas.
- ExcelLoader: leitura de workbook e seleção de sheet/colunas.
- ClipboardParser: detecção de tabulação, espaços e vírgulas, com atenção especial a separador decimal pt-BR.
- A camada Data trata coerção tabular e conversão para NumPy; DatasetValidator valida a representação de domínio: matrizes 2D, valores numéricos finitos, correspondência entre colunas e Variable, símbolos distintos e igualdade do número de observações.

Ambiguidade decimal: Quando vírgula puder significar delimitador de coluna ou separador decimal, o modo Auto deve propor uma interpretação, mas a interface deve permitir override explícito de Delimiter e Decimal separator.

8. Model Engine e interpretação de LaTeX

O Model Engine é o componente mais crítico. Sua responsabilidade é converter uma entrada LaTeX em uma ModelDefinition segura, determinística e verificável.

8.1 Pipeline normativa

1. Receber a string LaTeX e normalizar espaços/comandos permitidos.
2. Validar a forma geral variável_dependente = expressão e resolver o símbolo da variável dependente a partir do lado esquerdo.
3. Converter LaTeX para expressão SymPy através de LatexModelParser.
4. Validar funções, operadores e símbolos resultantes.
5. Identificar as variáveis independentes a partir da configuração do Dataset/GUI; a V1 exige exatamente uma, e x é apenas o símbolo padrão quando o usuário não define outro.
6. Extrair como parâmetros os símbolos livres distintos das variáveis independentes e das constantes matemáticas reconhecidas; na V1, a única variável dependente vem do lado esquerdo da equação.
7. Criar objetos Parameter com symbol, name opcional e defaults numéricos seguros; o parser mantém o mapeamento entre a notação LaTeX e a representação simbólica interna.
8. Gerar uma função NumPy através de sympy.lambdify.
9. Testar a função com valores de prova e verificar retorno numérico finito quando aplicável.
10. Expor a interpretação de volta à GUI antes de habilitar Fit.

8.2 Exemplo

```
Entrada LaTeX:
    T = A e^{-k t} + C

Representação simbólica esperada:
    A*exp(-k*t) + C

Variável dependente:
    T (ex.: name="Temperatura", unit="°C")

Variável independente:
    t (ex.: name="Tempo", unit="s")

Parâmetros de ajuste:
    A, k, C

Função numérica conceitual:
    f(t, A, k, C)
```

8.3 Regras de interpretação

- A V1 aceita modelos explícitos na forma variável_dependente = f(variável_independente, parâmetros), com exatamente uma variável independente e uma dependente. Internamente, ModelDefinition e Dataset usam coleções de variáveis para permitir evolução futura para múltiplas entradas/saídas. x e y são defaults de interface, não nomes obrigatórios.
- Expressões implícitas, sistemas de equações e derivadas diferenciais não fazem parte da V1.
- Símbolos matemáticos de Parameter e Variable devem ser validados por SymbolValidator antes de integrar o modelo. O validator aceita uma gramática atômica restrita, incluindo identificadores, letras gregas/comandos LaTeX reconhecidos e subscrito simples; operadores e expressões como A+B, x^2, sin(x) ou \frac{A}{B} não são símbolos válidos. O LatexModelParser permanece responsável por interpretar expressões completas e manter o mapeamento determinístico entre a entrada LaTeX, Parameter.symbol/Variable.symbol e a representação SymPy.
- Funções permitidas inicialmente: exp, log/ln, sin, cos, tan, sqrt e potências; ampliar somente com testes.
- Constantes matemáticas reconhecidas, como pi e e, não devem ser extraídas como parâmetros.
- Símbolos reservados e conflitos de nomes devem gerar erro de validação, nunca interpretação silenciosa.

8.4 Confirmação da interpretação

A GUI deve mostrar o modelo interpretado, as variáveis independente e dependente (symbol, name e unit quando disponíveis) e a lista de parâmetros antes do fit. O botão Fit permanece desabilitado quando a interpretação está inválida ou pendente.

Segurança: É proibido usar eval() diretamente sobre a expressão fornecida pelo usuário. A entrada passa pelo parser, pela validação simbólica e somente então por lambdify para avaliação numérica.

9. Fitting Engine

FitEngine define o contrato de ajuste. A implementação inicial recomendada é LmfitEngine, mantendo SciPy disponível como base numérica. O FitService recebe Dataset, ModelDefinition e parâmetros configurados e devolve FitResult.

```
abstract FitEngine
    fit(dataset: Dataset, model: ModelDefinition, parameters: list[Parameter]) -> FitResult

LmfitEngine(FitEngine)
    - cria lmfit.Parameters
    - configura min/max/vary
    - executa minimização
    - converte saída para FitResult
```

9.1 Initial guesses

Para modelos não lineares, os valores iniciais são parte do problema numérico. Modelos built-in podem registrar uma InitialGuessStrategy; modelos customizados recebem defaults e exigem revisão visual do usuário.

Modelo	Estratégia inicial sugerida
Linear	Regressão linear direta para a e b.
Exponencial	Assíntota para C, amplitude inicial para A e inclinação logarítmica aproximada para k.
Gaussiano	Offset pelo baseline, centro no máximo, amplitude pelo pico e sigma pela largura aproximada.
Customizado	Defaults neutros (ex.: 1) e edição manual obrigatoriamente disponível.

9.2 Limites e parâmetros fixos

Cada Parameter deve aceitar initial_value, min_value, max_value e vary. Os limites podem ser infinitos para representar ausência de restrição, enquanto initial_value e value ajustado devem ser finitos. A reconfiguração é completa: configure(...) substitui o estado configurável e limpa resultados de fit anteriores; configure() sem argumentos restaura os defaults preservando symbol.

10. Modelos padrão

Modelo	Template conceitual	Parâmetros
Linear	y = a x + b	a, b
Polinomial n	y = a_0 + a_1 x + ... + a_n x^n	a_0 ... a_n
Exponencial	y = A e^{B x} + C	A, B, C
Decaimento exponencial	y = A e^{-k x} + C	A, k, C

Modelo	Template conceitual	Parâmetros
Logarítmico	y = A ln(x) + B	A, B
Lei de potência	y = A x^n	A, n
Gaussiano	y = A exp(-(x-mu)^2/(2 sigma^2)) + C	A, mu, sigma, C
Logístico	y = L/(1+exp(-k(x-x0))) + C	L, k, x0, C

Os modelos padrão devem ser registrados no ModelRegistry preferencialmente como templates que resultem em ModelDefinition, evitando caminhos paralelos de fitting.

11. Interface gráfica Tkinter

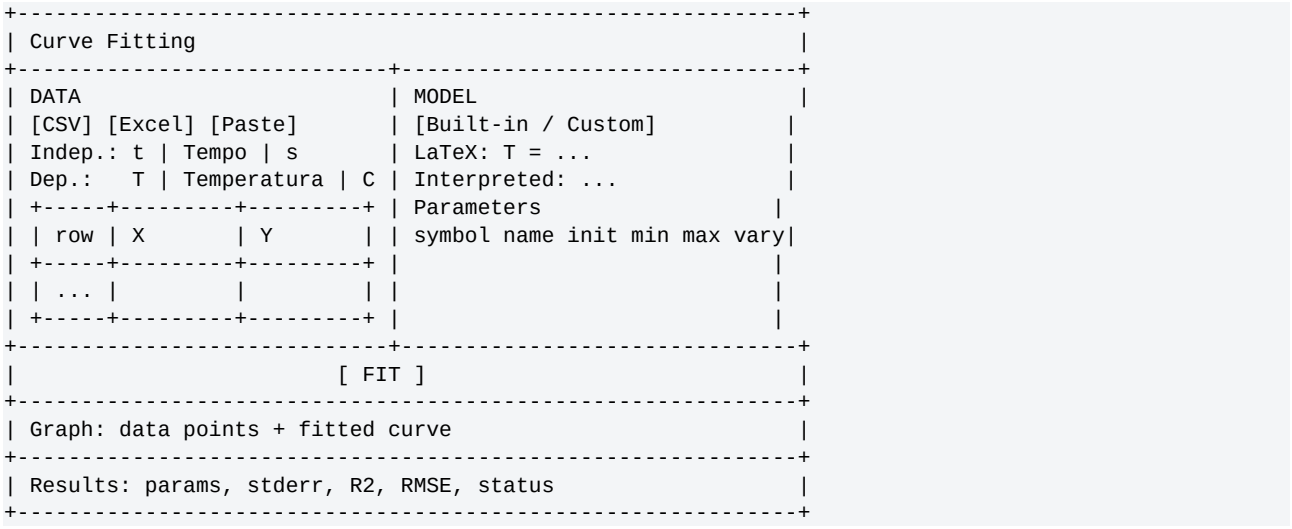
A janela principal deve ser composta por painéis independentes, cada um responsável por um aspecto da interação. Views não executam lógica científica.

Componente	Responsabilidade
MainWindow	Composição geral, menu, layout e estado global de interação.
DataPanel	Tabela de dados, importar, colar, limpar, selecionar colunas e editar metadados das Variable (symbol/name/unit).
ModelPanel	Escolher built-in ou editar LaTeX customizado; mostrar validação e associação dos símbolos às variáveis.
ParameterPanel	Exibir symbol e editar name/initial/min/max/vary dos parâmetros descobertos.
ResultsPanel	Exibir status, métricas, parâmetros e incertezas.
PlotPanel	Canvas Matplotlib para dados e curva ajustada.
Controllers	Traduzir eventos das views para chamadas aos services.

11.1 Grid de dados

A primeira implementação pode usar ttk.Treeview, com edição de células, seleção de linhas, Ctrl+V, Delete, Import CSV, Import Excel e Clear. A aplicação não deve tentar replicar uma planilha completa na V1.

11.2 Layout conceitual



12. Resultados, gráficos e métricas

- O gráfico deve sempre distinguir pontos experimentais da curva do modelo.
- A curva deve ser avaliada em x_smooth para apresentação visual, independentemente dos x experimentais.
- O FitResult armazena fitted_y nos x originais para resíduos e métricas.
- Métricas mínimas: RSS, RMSE e R^2; reduced chi-square somente quando sua interpretação for válida para o problema/configuração.
- Parâmetros devem exibir valor ajustado e stderr quando disponível.
- Falha de convergência deve produzir FitResult/erro controlado e mensagem no ResultsPanel.

13. Tratamento de erros e segurança

Exceção de domínio	Uso
DataImportError	Falha ao ler CSV/Excel ou conteúdo incompatível.
DataValidationError	Dados vazios, tamanhos diferentes, não numéricos ou insuficientes.
ModelParseError	LaTeX não pôde ser convertido para expressão válida.
ModelValidationError	Forma inválida, função/símbolo não permitido, conflito entre variável independente/dependente e parâmetros, ou associação de símbolos inconsistente.
ParameterError	Initial/min/max inconsistentes.
FitError	Falha numérica, não convergência ou avaliação inválida.
PlotError	Erro ao renderizar resultados.

- Não exibir traceback bruto ao usuário final; registrar detalhes em log quando logging for incorporado.
- Nunca executar código arbitrário vindo do campo de modelo.
- Validar domínio de funções, por exemplo log(x) para x>0, e comunicar valores inválidos.
- Tratar overflow/underflow e NaN/Inf durante a avaliação do modelo.
- Conservar a aplicação funcional após um fit que falhou.

14. Estratégia de testes

O desenvolvimento deve seguir do núcleo para a GUI. Parser e fitting devem ser testados sem interface gráfica.

Camada	Testes mínimos
domain	Construção, invariantes e serialização simples dos objetos.
data	CSV, XLSX, clipboard por tab/espaço/vírgula, decimal pt-BR, NaN/Inf, colunas inválidas.
models	LaTeX válido, inválido, funções, subscritos, parâmetros, constantes, símbolos reservados.
fitting	Dados sintéticos com solução conhecida, limites, parâmetro fixo, erro de convergência.
statistics	RSS, RMSE, R^2 e casos extremos.
integration	Dataset + ModelDefinition + FitEngine -> FitResult.
gui	Smoke tests de inicialização e controllers; evitar depender de GUI para validar ciência.

Critério de regressão: Sempre que um bug de parsing ou fitting for corrigido, adicionar um teste que reproduza o caso antes da correção.

15. Dependências e tecnologias

Tecnologia	Papel	Status arquitetural
Python 3.12+	Linguagem base	Recomendado; validar compatibilidade do ambiente final.
Tkinter / ttk	GUI desktop	Obrigatório pelo escopo atual.
NumPy	Matrizes canônicas do Dataset, álgebra numérica e avaliação vetorizada	Core do domínio científico.
Pandas	Importação, seleção e manipulação tabular antes da conversão para NumPy	Infraestrutura de dados; não vazar para domain.
openpyxl	Leitura de XLSX	Infraestrutura.
SymPy	Representação simbólica e lambdify	Core do Model Engine.
Lark / parser SymPy	Backend de parsing LaTeX	Encapsulado por LatexModelParser.
SciPy	Otimização numérica	Base científica.
LMFIT	Parâmetros e fitting não linear	Implementação inicial recomendada de FitEngine.
Matplotlib	Plotagem incorporada ao Tkinter	Output visual.
pytest	Testes	Recomendado.

Regra de dependência: Nenhum módulo da camada domain deve importar Tkinter, Pandas, Matplotlib ou

LMFIT. Dependências externas ficam nas bordas/engines da arquitetura.

16. Fluxos principais

16.1 Importação e preparação de dados

```
User -> DataPanel -> DataController -> DataService
     -> CSVLoader / ExcelLoader / ClipboardParser
     -> DataValidator -> Dataset -> estado da aplicação
```

16.2 Construção de modelo customizado

```
User LaTeX -> ModelPanel -> ModelController -> ModelService
     -> LatexModelParser -> SymPy Expr -> ParameterExtractor
     -> ModelDefinition -> GUI mostra interpretação + parâmetros
```

16.3 Fitting

```
Dataset + ModelDefinition + Parameter settings
     -> FitController -> FitService -> FitEngine
     -> LmfitEngine -> Statistics -> FitResult
     -> ResultsPanel + Plotter -> PlotPanel
```

17. Roadmap de implementação

Fase	Entrega
Fase 1 - Domain	Dataset, Variable, Parameter, ModelDefinition, FitResult, validadores e exceções.
Fase 2 - Model Engine	LatexModelParser, extração de parâmetros, lambdify e ModelRegistry.
Fase 3 - Fitting	FitEngine, LmfitEngine, statistics, dados sintéticos e testes.
Fase 4 - Data Engine	CSV, XLSX, clipboard e validação.
Fase 5 - Services	DataService, ModelService e FitService.
Fase 6 - GUI estrutural	MainWindow, panels, widgets e controllers.
Fase 7 - Integração	Fluxo ponta a ponta e tratamento de erros.
Fase 8 - Polimento	UX, responsividade, logs, exportações e empacotamento.

Ordem recomendada: Não iniciar pela montagem completa da GUI. Primeiro validar Dataset -> LaTeX -> ModelDefinition -> FitResult com testes automatizados; depois acoplar Tkinter.

18. Regras de conformidade e controle de mudanças

Esta seção torna o documento uma baseline de verificação. Todo componente implementado deve ser confrontado com as regras abaixo.

11. Antes de criar ou modificar um módulo estrutural, verificar seu pacote e responsabilidade neste documento.
12. Se o código proposto divergir da baseline, a divergência deve ser informada explicitamente ao responsável pelo projeto antes de ser tratada como solução definitiva.

13. A análise deve identificar: (a) o que diverge; (b) por que diverge; (c) impacto; (d) recomendação: alterar código ou revisar documentação.
14. Não alterar silenciosamente a arquitetura apenas por conveniência local de implementação.
15. Quando uma mudança arquitetural for aprovada, atualizar a versão deste documento e registrar a decisão.
16. Refatorações internas que preservem contratos e responsabilidades não exigem mudança de baseline, mas devem manter os testes.

18.1 Classificação de divergências

Nível	Exemplo	Ação
Baixa	Nome de método/classe diferente, mesma responsabilidade e contrato.	Registrar na revisão; normalmente adaptar documentação somente se persistente.
Média	Mudança de dependência, novo serviço, reorganização de pacote.	Apresentar antes de consolidar; decidir código x documento.
Alta	GUI passa a executar lógica científica; eliminação do Model Engine; uso de eval; mudança do modelo de domínio.	Bloquear como baseline violation até decisão explícita.

19. Checklist de verificação

[ ] A GUI não contém lógica de parsing ou otimização.

[ ] Dataset usa matrizes NumPy 2D canônicas e não depende de Pandas/Tkinter.

[ ] LaTeX é convertido para representação SymPy antes do fitting.

[ ] Não existe eval() de entrada matemática do usuário.

[ ] A variável independente configurada é distinguida dos parâmetros de forma determinística; x é somente o default.

[ ] Modelos built-in e customizados resultam em ModelDefinition.

[ ] Parâmetros possuem symbol atômico validado, name opcional, initial/min/max/vary, stderr e unit opcional.

[ ] FitEngine possui contrato independente da implementação LMFIT.

[ ] FitResult concentra resultados, métricas e arrays de saída.

[ ] Data Engine trata CSV, Excel e clipboard separadamente.

[ ] Erros de domínio são controlados e convertidos em mensagens de UI.

[ ] Parser e fitting possuem testes independentes de Tkinter.

[ ] Dados sintéticos recuperam parâmetros conhecidos dentro de tolerância.

[ ] Gráfico usa Dataset + FitResult, não recalcula fitting.

[ ] Toda divergência arquitetural relevante foi explicitamente avaliada.

20. Arquitetura completa do software

O diagrama abaixo consolida camadas, responsabilidades e dependências esperadas. Setas indicam fluxo/dependência principal, não necessariamente chamadas síncronas exclusivas.

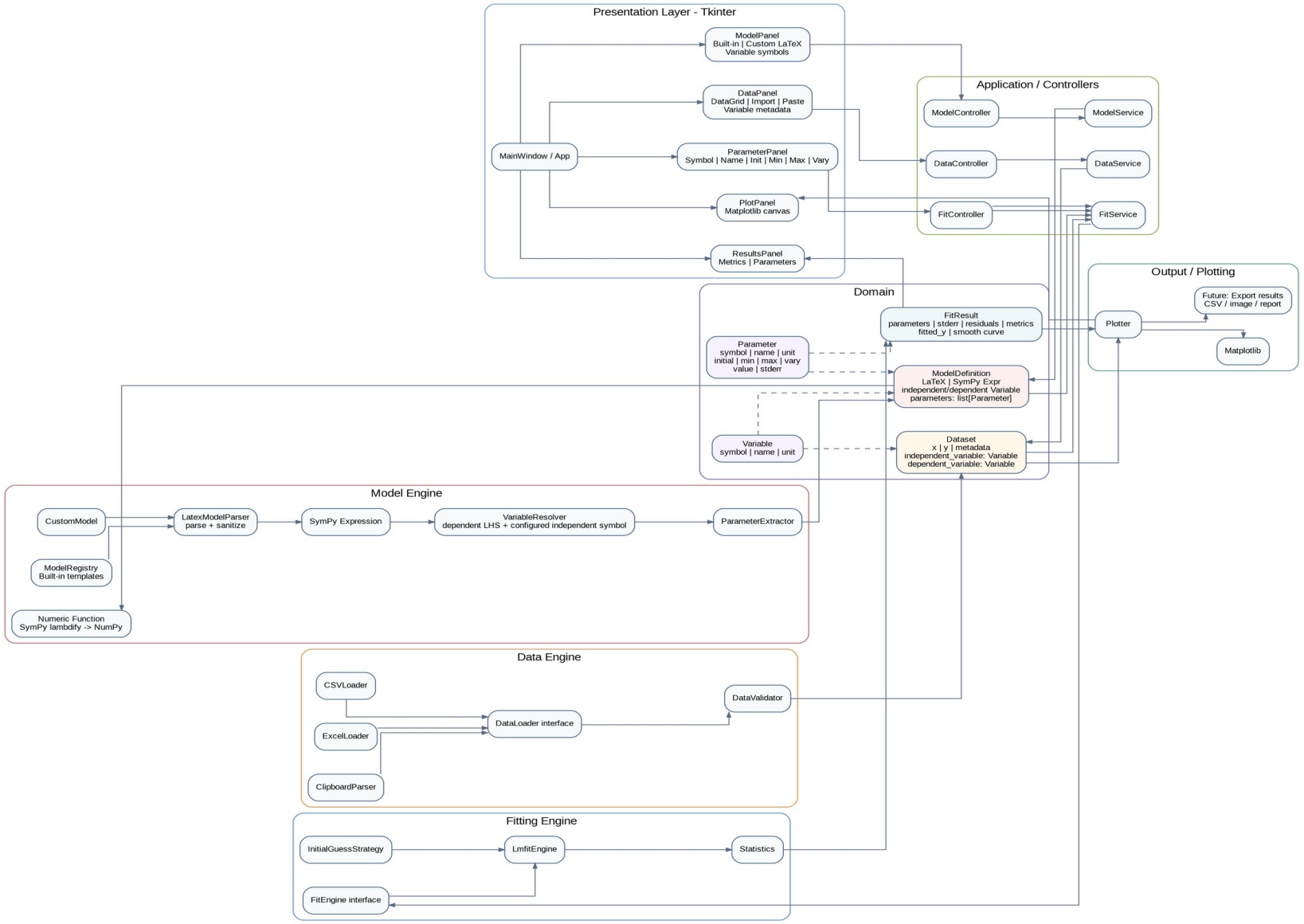


Figura 1 - Arquitetura lógica completa da aplicação de curve fitting.

20.1 Dependências permitidas

- gui -> controllers -> services.
- services -> domain e interfaces de engines.
- data -> domain.Dataset.
- models -> domain.ModelDefinition / Parameter / Variable e SymPy.
- fitting -> domain + biblioteca numérica.
- plotting -> Dataset/FitResult + Matplotlib.
- domain não depende de gui, data, fitting, plotting ou frameworks externos de interface.

20.2 Dependências proibidas

- domain -> Tkinter/Pandas/Matplotlib/LMFIT.
- views -> scipy.optimize ou lmfit diretamente.
- plotter -> executar novo fit.
- CSVLoader/ExcelLoader -> atualizar widgets Tkinter.
- LatexModelParser -> exibir messagebox ou alterar estado da GUI.

21. Decisões em aberto e extensões futuras

Tema	Decisão inicial	Possível evolução
Parser LaTeX	Encapsular backend e começar com parser SymPy/Lark validado por testes.	Gramática própria ou MathLive/webview se a UX exigir.
Fitting	LMFIT como engine inicial.	SciPyFitEngine, robust loss, BayesianFitEngine.
Planilha	ttk.Treeview.	Widget tabular especializado se edição avançada se tornar requisito.
Threading	Fit síncrono nos primeiros testes de núcleo.	Worker thread/queue para datasets/modelos lentos na GUI.
Exportação	Não central na V1.	CSV de parâmetros, PNG/SVG, relatório PDF.
Modelos	V1: uma variável independente e uma dependente; Dataset/ModelDefinition já usam coleções e matrizes 2D canônicas.	Múltiplas variáveis independentes/dependentes, global fit, múltiplos datasets, modelos implícitos e ODEs.
Persistência	Sessão em memória.	Salvar/carregar projeto, modelos favoritos e presets.

22. Referências técnicas

- SymPy - documentação de parsing LaTeX, representação simbólica e lambdify.
- LMFIT - documentação de Model, Parameters, minimização, limites e incertezas.
- SciPy - scipy.optimize, least_squares e curve_fit.
- NumPy - arrays, broadcasting e operações vetorizadas.
- Pandas / openpyxl - importação e manipulação tabular CSV/XLSX antes da conversão para NumPy.
- Matplotlib - integração com Tkinter via FigureCanvasTkAgg.

- Python tkinter / ttk - widgets desktop e event loop.

Anexo A - Critério de aceite da primeira vertical funcional

A primeira vertical funcional será considerada arquiteturalmente válida quando o programa, ainda que com GUI simples, conseguir executar de ponta a ponta o seguinte cenário sem atalhos que violem a separação de camadas:

17. Usuário cola ou importa um dataset com duas colunas.
18. Aplicação constrói Dataset validado.
19. Usuário seleciona um modelo built-in ou informa T = A e^{-k t} + C em LaTeX, podendo usar símbolos diferentes dos defaults x/y.
20. Aplicação reconhece t e T como Variable, mostra A, k e C como Parameter e permite editar name/initial/min/max/vary dos parâmetros.
21. FitService executa o FitEngine sem a view importar LMFIT/SciPy.
22. FitResult retorna parâmetros, métricas, resíduos e curva suave.
23. ResultsPanel mostra valores e PlotPanel exibe dados + curva.
24. Um conjunto de testes de integração reproduz o mesmo fluxo sem abrir Tkinter.

Anexo B - Registro inicial de decisões arquiteturais

ID	Decisão	Status
ADR-001	Separar GUI e núcleo científico	Aprovado na baseline v1.1
ADR-002	Usar LaTeX apenas como entrada/apresentação e SymPy como representação interna	Aprovado na baseline v1.1
ADR-003	Encapsular parsing em LatexModelParser	Aprovado na baseline v1.1
ADR-004	Não utilizar eval() em entrada do usuário	Obrigatório
ADR-005	Adotar FitEngine abstrato e LmfitEngine inicial	Aprovado na baseline v1.1
ADR-006	Modelos built-in e customizados convergem para ModelDefinition	Aprovado na baseline v1.1
ADR-007	Introduzir Variable para representar as grandezas independente e dependente com symbol/name/unit	Aprovado na baseline v1.1
ADR-008	Separar identidade matemática e descrição humana: Parameter.symbol é matemático/LaTeX; Parameter.name é descritivo opcional	Aprovado na baseline v1.1
ADR-009	Tratar x e y apenas como símbolos padrão; permitir símbolos configuráveis para os eixos sem	Aprovado na baseline v1.1

ID	Decisão	Status
	transformá-los em Parameter	
ADR-010	Definir o contrato de Parameter: symbol imutável; configure(...) substitui toda a configuração, restaura defaults quando vazio e invalida o fit anterior via clear_fit_result(); alterações inválidas são atômicas.	Aprovado na baseline v1.2
ADR-011	Introduzir TextValidator e SymbolValidator reutilizáveis. ParameterValidator compõe os validadores genéricos e Parameter depende apenas de ParameterValidator. SymbolValidator aceita somente símbolos matemáticos atômicos de gramática restrita; expressões completas permanecem no Model Engine.	Aprovado na baseline v1.3
ADR-012	Adotar Dataset com representação NumPy canônica sempre 2D: linhas são observações e colunas são variáveis; independent_variables/dependent_variables são coleções normalizadas para tuple. A API aceita uma Variable isolada ou Sequence[Variable]. DatasetValidator usa mensagens genéricas nos métodos reutilizáveis, sem field_name decorativo. Pandas permanece na camada Data e é convertido para ndarray antes do domínio. A V1 limita a cardinalidade a 1+1 por política.	Aprovado na baseline v1.4